

Mining Software Repositories

John Businge

References



The Road Ahead for Mining Software Repositories

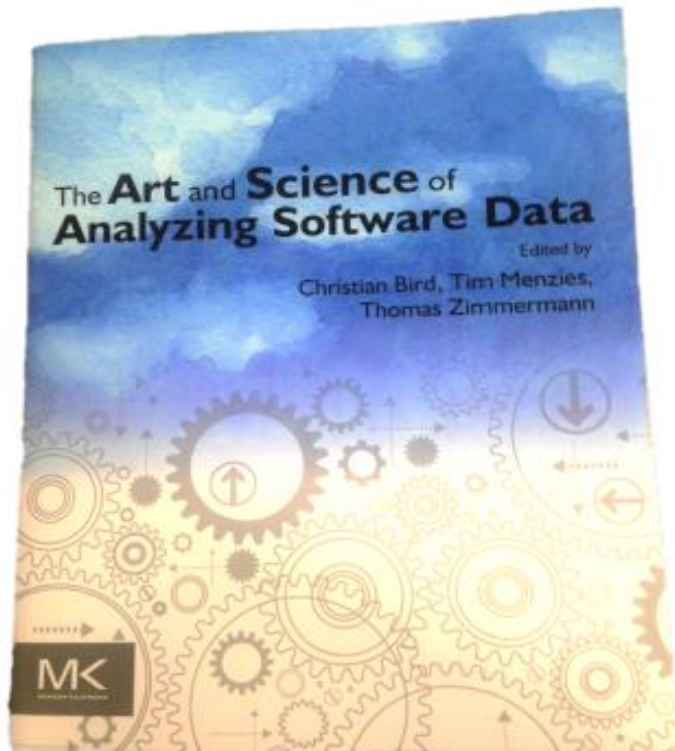
Ahmed E. Hassan
Software Analysis and Intelligence Lab (SAIL)
School of Computing, Queen's University, Canada
ahmed@cs.queensu.ca

Software Intelligence: The Future of Mining Software Engineering Data

Ahmed E. Hassan
School of Computing
Queen's University
Kingston, ON, Canada
ahmed@cs.queensu.ca

Tao Xie
Department of Computer Science
North Carolina State University
Raleigh, NC, USA
xie@csc.ncsu.edu

More References



Acknowledgement



Ahmed E. Hassan

Queen's University

www.cs.queensu.ca/~ahmed

ahmed@cs.queensu.ca

Tao Xie

North Carolina State University

www.csc.ncsu.edu/faculty/xie

xie@csc.ncsu.edu

Bram Adams

Polytechnique Montréal, Canada

<http://mcis.polymtl.ca/index.html>

lab.mcis@gmail.com

With updates by **John Businge** from the University of Nevada Las Vegas

Lecture Goals



Learn about:

- Classic and notable research and researchers in mining SE data
- Data mining and data processing techniques and how to apply them to SE data
- Risks in using SE data due to noise and incompleteness

After the lecture and lab, you should be able to:

- Retrieve software evolution data from Git and GitHub
- Trace a pull request to its commits and merge commit
- Mine repository history to understand a software change
- Use repository evidence to support a reengineering task

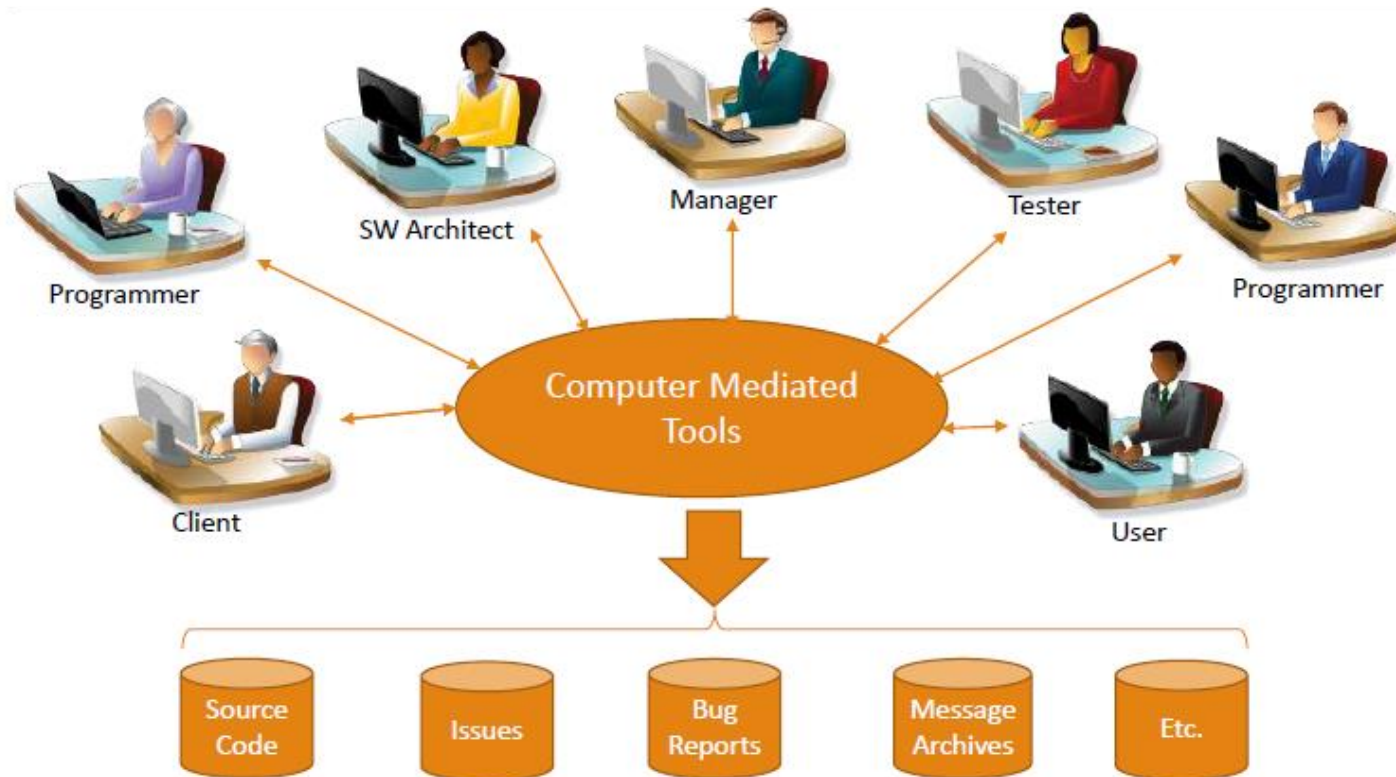
Why mine SE data?

- **SE data can be used to:**

- Gain empirically-based understanding of software development
- Predict, plan, and understand various aspects of a project
- Support future development and project management activities



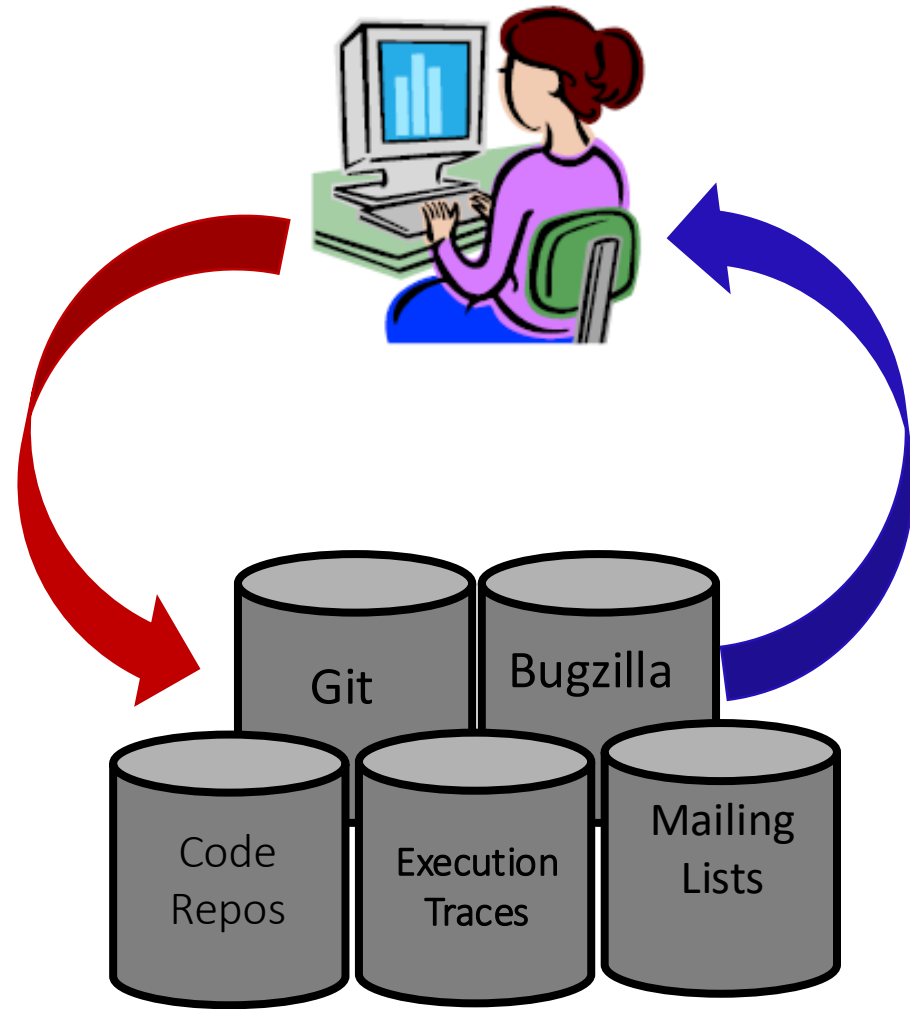
How is SE Data generated?



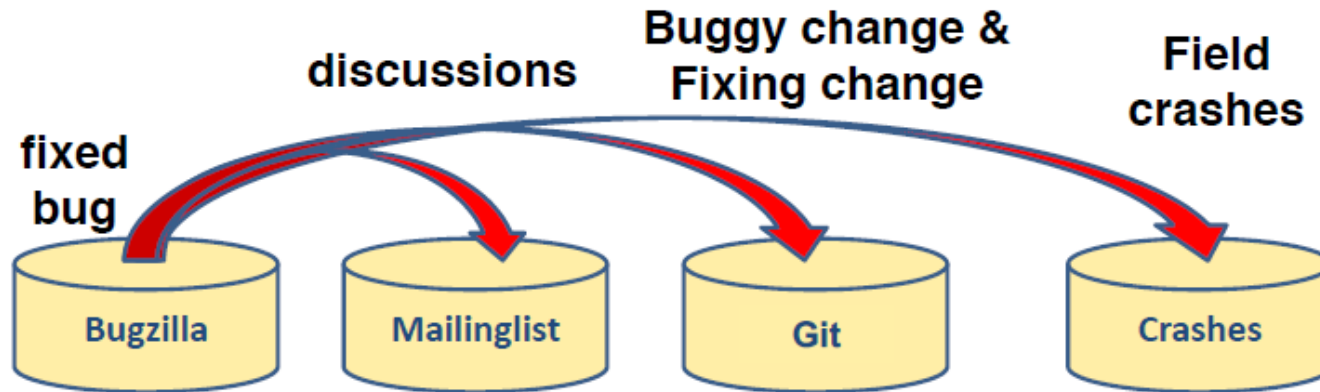
Current and historical artifacts and interactions are registered in software repositories

What is MSR?

- Transforming static record keeping SE into active data
- Making SE data actionable by uncovering patterns and trends



MSR researchers analyze and cross-link repositories



New bug report
Estimate fix effort
Suggest experts and fix!

Study Outline



- **Part I:** What can we learn from SE data?
 - A sample of notable findings for different SE data types
- **Part II:** How can we mine SE data?
 - Understand the structure of SE data

MSR studies – Bugs – Part I

Using imports to predict Bugs

71% of files that import **compiler** packages,
had to be fixed later on.

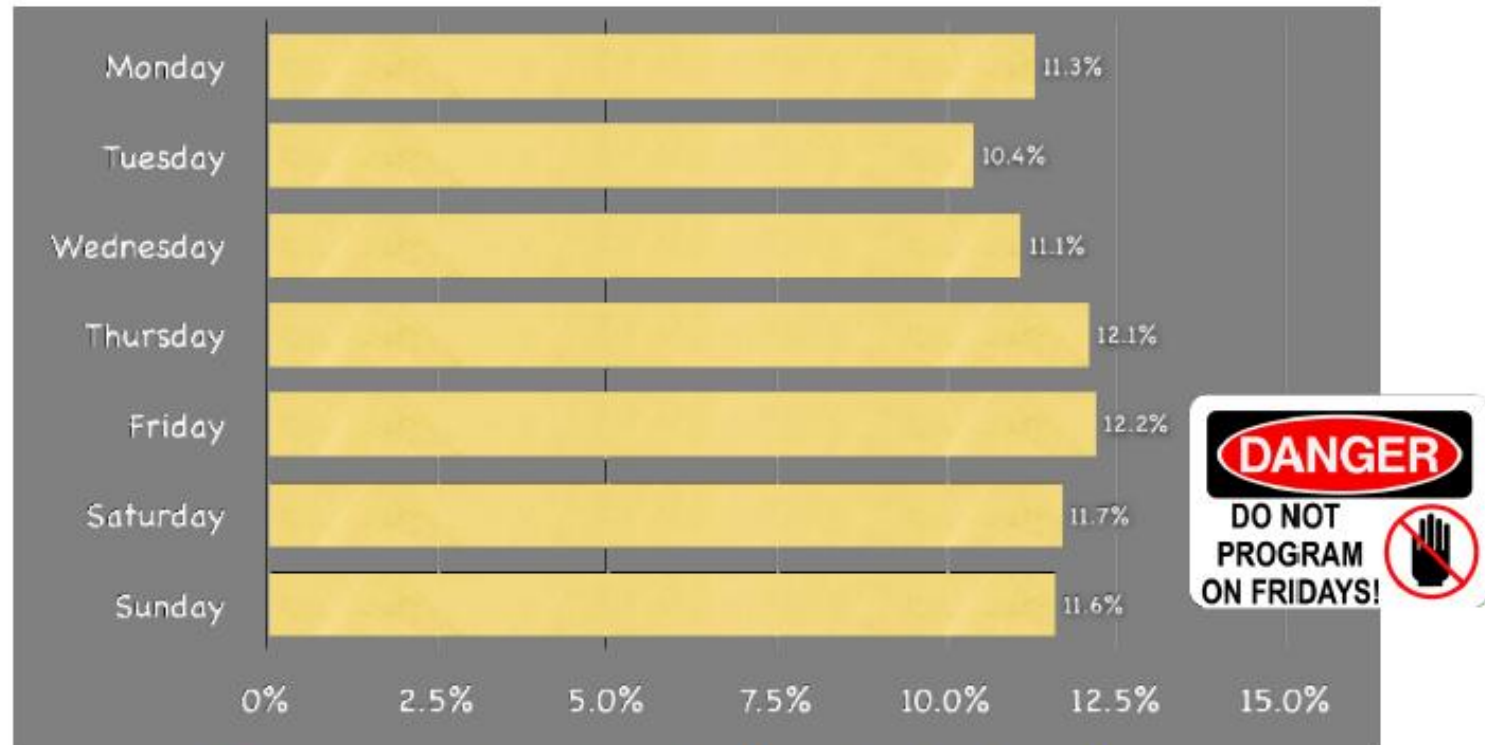
```
import org.eclipse.jdt.internal.compiler.lookup.*;
import org.eclipse.jdt.internal.compiler.*;
import org.eclipse.jdt.internal.compiler.ast.*;
import org.eclipse.jdt.internal.compiler.util.*;
...
import org.eclipse.pde.core.*;
import org.eclipse.jface.wizard.*;
import org.eclipse.ui.*;
```

[Schröter et al. 06]

14% of all files that import **ui** packages, had
to be fixed later on.

MSR studies - Bugs

Do not program on Friday ;-)

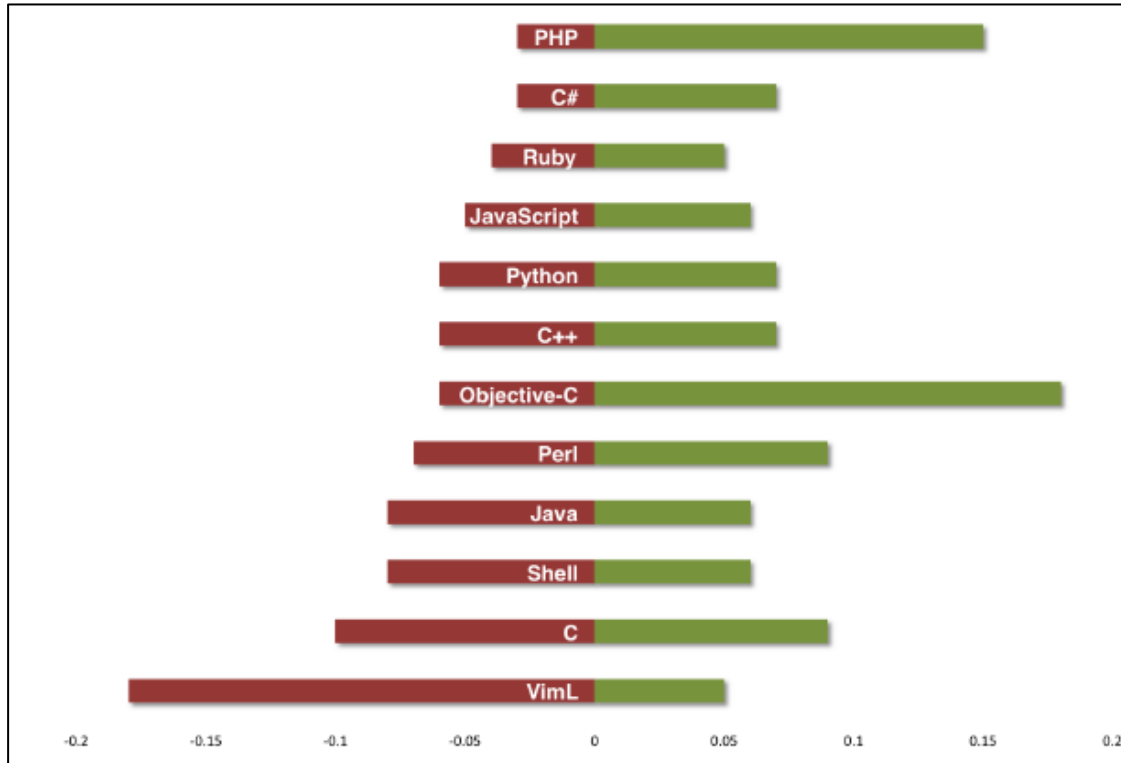


Percentage of bug-introducing changes for eclipse

[Zimmermann et al. 05]

MSR studies – Sentiment Analysis

Anger vs. Joy



How they stack up?

- **PHP, Object-C, and C#** are net positive
- **Java, Shell, and C** are fairly even while **VimL** is just bad news.

[Doll and Grigoriuk, 2012]

MSR studies – Sentiment Analysis



10/23/12 3:56 AM

Disable 'showmatch' option Matching parens
are highlighted even without this option;
what it does is jump the cursor to the
matching paren which is [REDACTED] insane.



10/23/12 3:28 AM

styles everywhere, tutorial for first user
login, fixing some css [REDACTED]



10/23/12 2:57 AM

[REDACTED] again



10/23/12 2:30 AM

more [REDACTED]



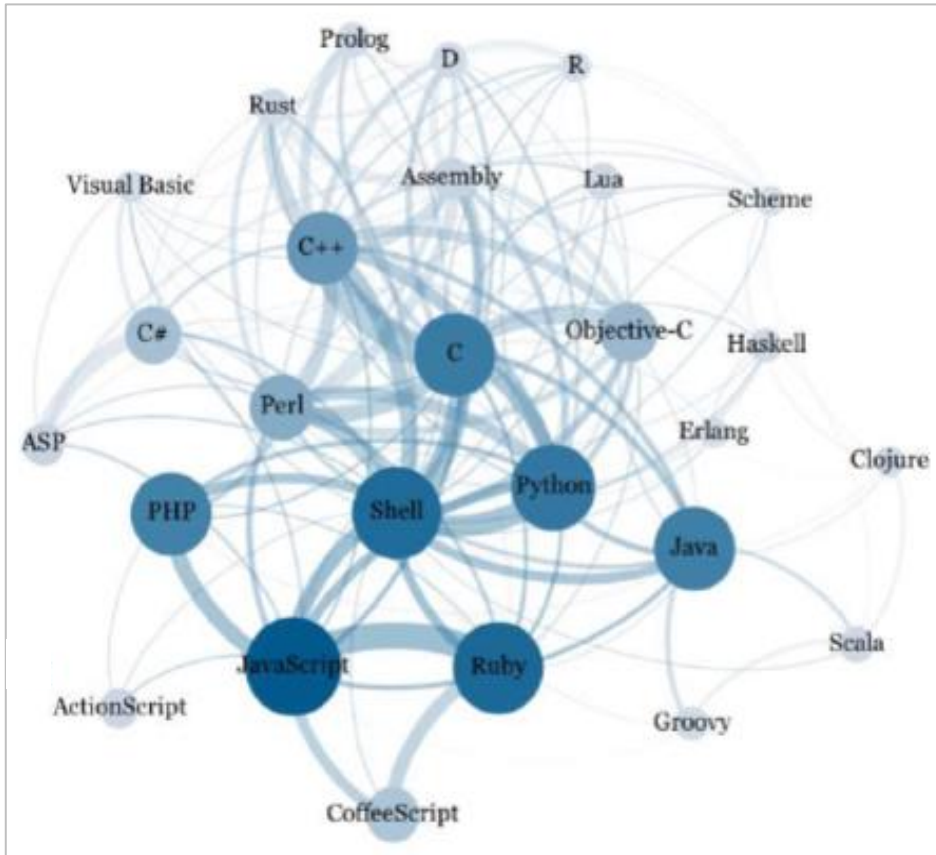
10/23/12 2:29 AM

Security [REDACTED] worked out



MSR studies – Programming languages

Programming language relations



A **Ruby** programmers is **very likely to know Javascript**, while a **Perl** programmer is not

Java is a popular programming language but stands primarily alone

<https://github.com/mjwillson/ProgLangVisualise>

MSR studies – Changes by programmers

Programming language relations

[Zimmermann et al., 2005]

Mining Version Histories to Guide Software Changes

A) The user inserts a new preference into the field fKeys[]

B) ROSE suggests locations for further changes, e.g. the function initDefaults()

Symbol	File	Support	Confidence
initDefaults(PreferenceStore store)	ComparePreferencePage.java	7	1.0
org.eclipse.compare.plugin.properties	plugin.properties	0	0.875
org.eclipse.compare.subnotes_compare.html	subnotes_compare.html	0	0.75
TextMergeViewer(Composite parent, int style, CompareConfiguration configuration)	TextMergeViewer.java	6	0.75
propertyChanged(PropertyChangeEvent event)	TextMergeViewer.java	6	0.75
createGeneralPage(Composite parent)	ComparePreferencePage.java	5	0.625
createTextComparePage(Composite parent)	ComparePreferencePage.java	5	0.625
handleDispose(DisposeEvent event)	TextMergeViewer.java	4	0.5

After the programmer has made some changes to the source (above), **ROSE** suggests locations (below) where, in similar transactions in the past, further changes were made

- Suggests and predicts likely changes
- Prevents errors due to incomplete changes

How can we mine SE Data – Part II

Repositories of Repositories



Feb 2026:
180M developers
630M repositories
4.3M AI projects



January 2020:
430K repositories
3.7 Million Users



April 2019
28 Millinon repositories
10 Million Users



April 2019
28K projects



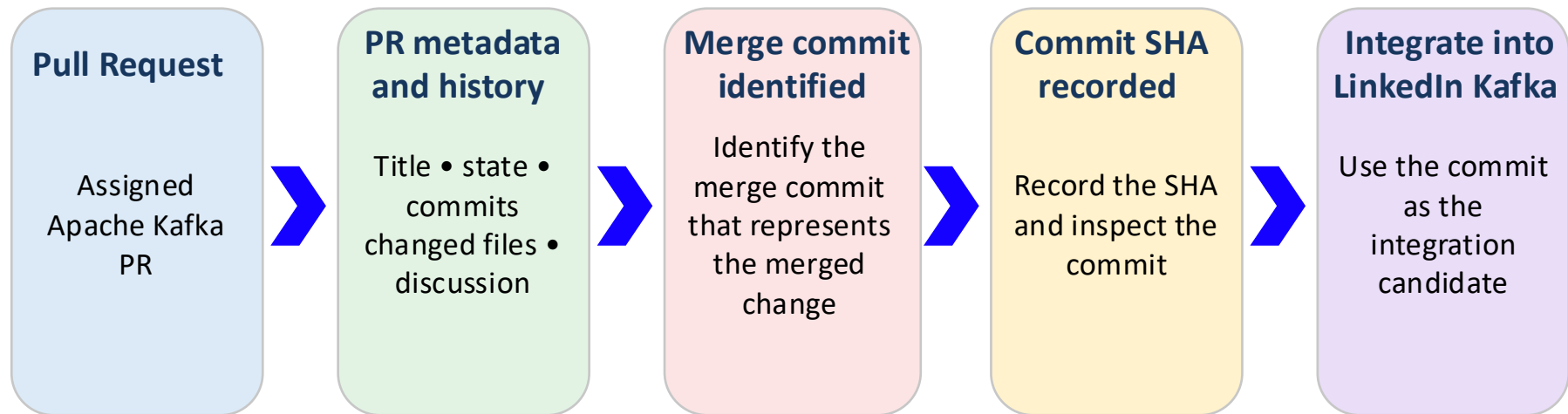
How can we mine SE Data form GitHub



From a Pull Request to the Merge Commit



Given an Apache Kafka pull request, how do we identify the commit to integrate?



How you obtain the evidence is your choice:
Git • GitHub UI • REST API • GraphQL API • Scripts

Goal: systematically recover the repository information needed for integration.

How can we mine SE Data



Search entire site...

Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient **staging areas**, and **multiple workflows**.



Easiest to obtain local copy (distributed version control!), and has distinction between authors and committers, but ... branches can be pain to analyse

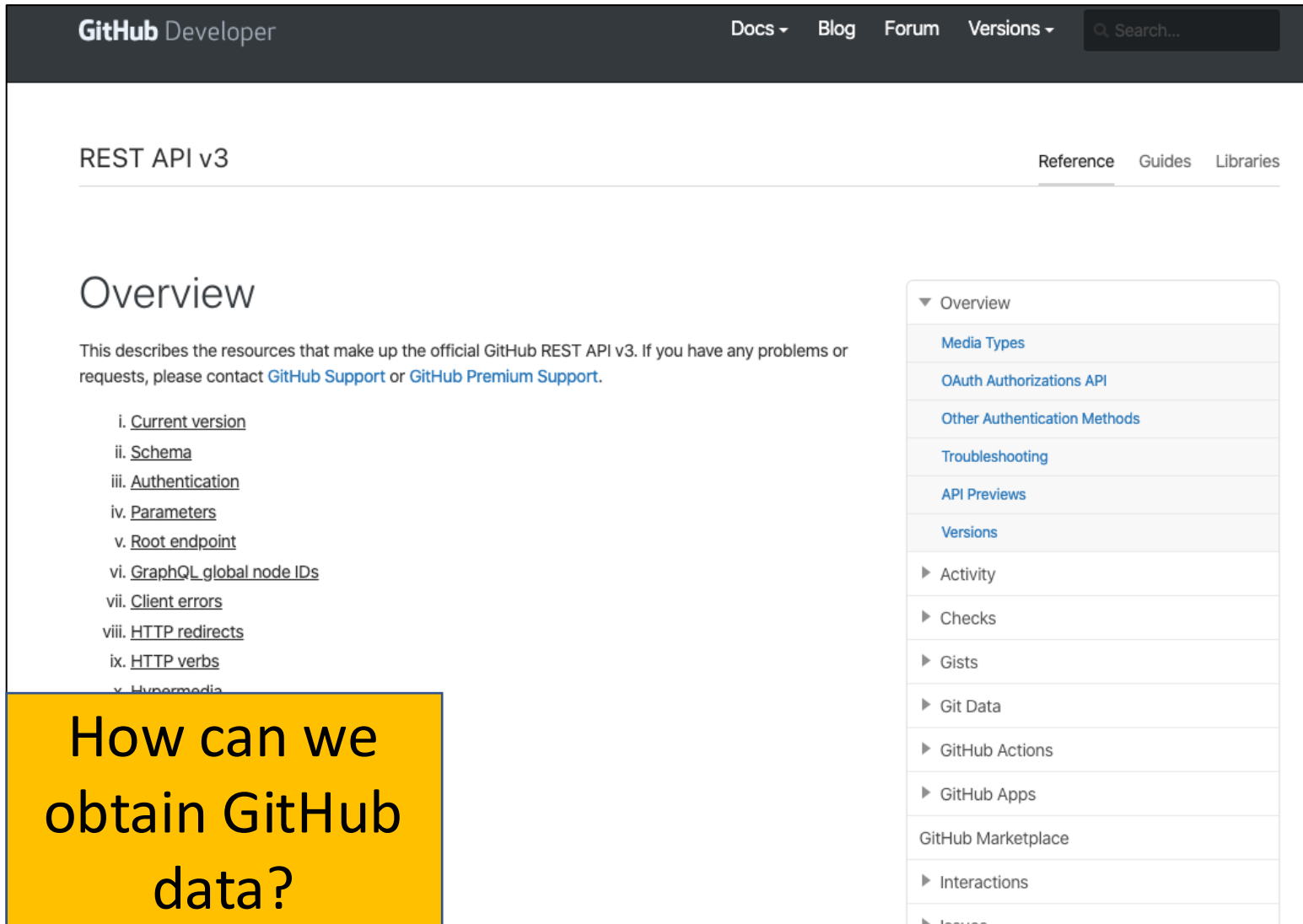


How can we mine SE Data

The image is a screenshot of the GitHub homepage. At the top, there is a search bar with the GitHub logo and the text "Search GitHub". To the right of the search bar are links for "Explore", "Gist", "Blog", and "Help". Further right is the user profile "bramadams" with a plus sign, a notification bell, a gear icon for settings, and a mail icon. Below the top navigation bar, there is a secondary bar with the "bramadams" profile, a "News Feed" icon, a "Pull Requests" icon, and an "Issues" icon. The main content area features a "GitHub Bootcamp" section with four numbered steps: 1. Set up Git (A quick guide to help you get started with Git.), 2. Create repositories (Repositories are where you'll work and collaborate on projects.), 3. Fork repositories (Forking creates a new, unique project from an existing one.), and 4. Work together (Send pull requests, follow friends. Star and watch projects.). Below the bootcamp section, there is a yellow text box with the text "Access to millions of Git-based projects via GitHub". To the right of the text box, there is a notification for "Better Word Highlighting in Diffs" and a section titled "Repositories you contribute to" listing "inukshuk/jekyll-scholar" with 152 stars and "smcintosh/moosetracks" with 1 star.

Access to millions of Git-based projects via GitHub

How can we mine GitHub Data



The screenshot shows the GitHub Developer REST API v3 Overview page. The page has a dark header with the GitHub Developer logo and navigation links: Docs, Blog, Forum, Versions, and a search bar. Below the header, the page title is 'REST API v3' with tabs for Reference, Guides, and Libraries. The main content area is titled 'Overview' and contains a paragraph describing the resources of the official GitHub REST API v3. Below this paragraph is a list of links: i. [Current version](#), ii. [Schema](#), iii. [Authentication](#), iv. [Parameters](#), v. [Root endpoint](#), vi. [GraphQL global node IDs](#), vii. [Client errors](#), viii. [HTTP redirects](#), ix. [HTTP verbs](#), and x. [Hypertext](#). On the right side of the page is a sidebar with a table of contents. A yellow callout box is overlaid on the bottom left of the page, containing the text 'How can we obtain GitHub data?'.

GitHub Developer

Docs ▾ Blog Forum Versions ▾

REST API v3

Reference Guides Libraries

Overview

This describes the resources that make up the official GitHub REST API v3. If you have any problems or requests, please contact [GitHub Support](#) or [GitHub Premium Support](#).

- i. [Current version](#)
- ii. [Schema](#)
- iii. [Authentication](#)
- iv. [Parameters](#)
- v. [Root endpoint](#)
- vi. [GraphQL global node IDs](#)
- vii. [Client errors](#)
- viii. [HTTP redirects](#)
- ix. [HTTP verbs](#)
- x. [Hypertext](#)

▼ Overview

- [Media Types](#)
- [OAuth Authorizations API](#)
- [Other Authentication Methods](#)
- [Troubleshooting](#)
- [API Previews](#)
- [Versions](#)
- ▶ Activity
- ▶ Checks
- ▶ Gists
- ▶ Git Data
- ▶ GitHub Actions
- ▶ GitHub Apps
- GitHub Marketplace
- ▶ Interactions
- ▶ Issues

How can we obtain GitHub data?

How can we mine GitHub Data

GitHub Developer

Docs ▾ Blog Forum Versions ▾

REST API v3

Reference Guides Libraries

Overview

This describes the resources that make up the official GitHub REST API v3. If you have any problems or requests, please contact [GitHub Support](#) or [GitHub Premium Support](#).

- i. [Current version](#)
- ii. [Schema](#)
- iii. [Authentication](#)
- iv. [Parameters](#)
- v. [Root endpoint](#)
- vi. [GraphQL global node IDs](#)
- vii. [Client errors](#)
- viii. [HTTP redirects](#)
- ix. [HTTP verbs](#)
- x. [Hypertext](#)

▼ Overview
Media Types
OAuth Authorizations API
Other Authentication Methods
Troubleshooting
API Previews
Versions
► Activity

How can we obtain GitHub data?

There are rate-limit issues!
You are only allowed only a limited number of GitHub requests per hour

How can we mine GitHub Data

GitHub GraphQL API documentation

To create integrations, retrieve data, and automate your workflows, use the GitHub GraphQL API. The GitHub GraphQL API offers more precise and flexible queries than the GitHub REST API.

Overview

Start here [View all →](#)

Forming calls with GraphQL

Learn how to authenticate to the GraphQL API, then learn how to create and run queries and mutations.

Introduction to GraphQL

Learn useful terminology and concepts for using the GitHub GraphQL API.

Using GraphQL Clients

You can run queries on real GitHub data using various GraphQL clients and libraries.

Popular

Explorer

Public schema

Download the public schema for the GitHub GraphQL API.

Using pagination in the GraphQL API

Learn how to traverse data sets using cursor based pagination with the GraphQL API.

How can we
obtain GitHub
data?

There are rate-limit issues!
Use GraphQL API
Rate limit is less of an issue

What Can We Mine for the Project?

A commit SHA is only the beginning

PR

What was the purpose of the change?

COMMITTS

Which commits belong to the PR?

FILES

Which files, classes, and methods changed?

TESTS

Were tests added or modified?

HISTORY

How did the relevant code evolve?

VARIANTS

How has the target diverged from the source?

Example evidence

PR #16221

✓ merged

Changed files

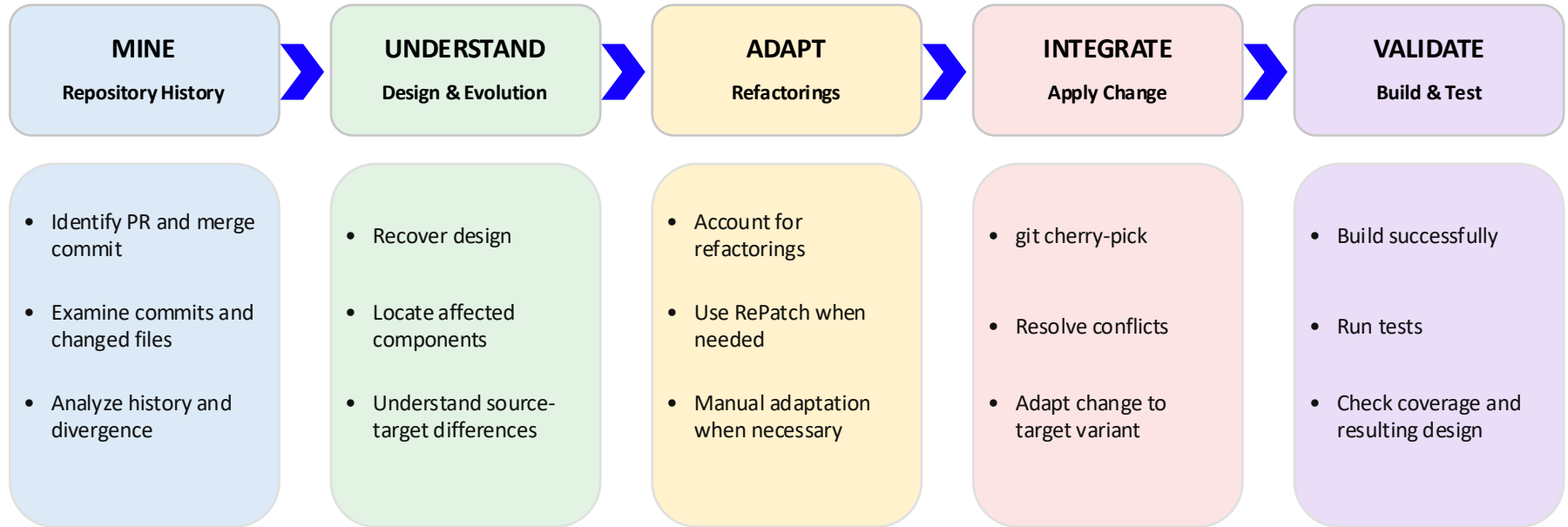
- LeaderElection.java
- KafkaMetrics.java
- LeaderElectionTest.java

Commits

- implementation
- refactor
- merge commit

Use repository history as evidence for understanding the change, not only for locating it.

How the Labs Fit Together



The labs are parts of one reengineering workflow.